PRACTICAL-6

AIM: Load data from JSON files with the Fetch API and render it dynamically with search, filter, sort and pagination using modular JavaScript.

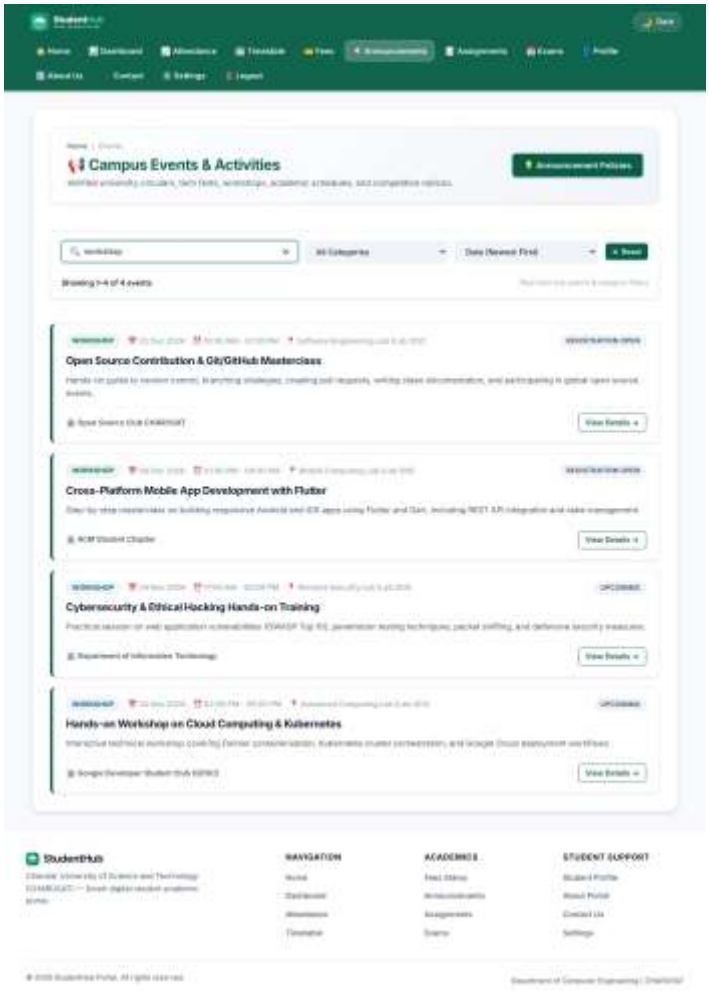
SOURCE CODE :

https://github.com/Aarav280907/WEB/tree/main/Practical(1-4)(Studenthub)/Practical-6

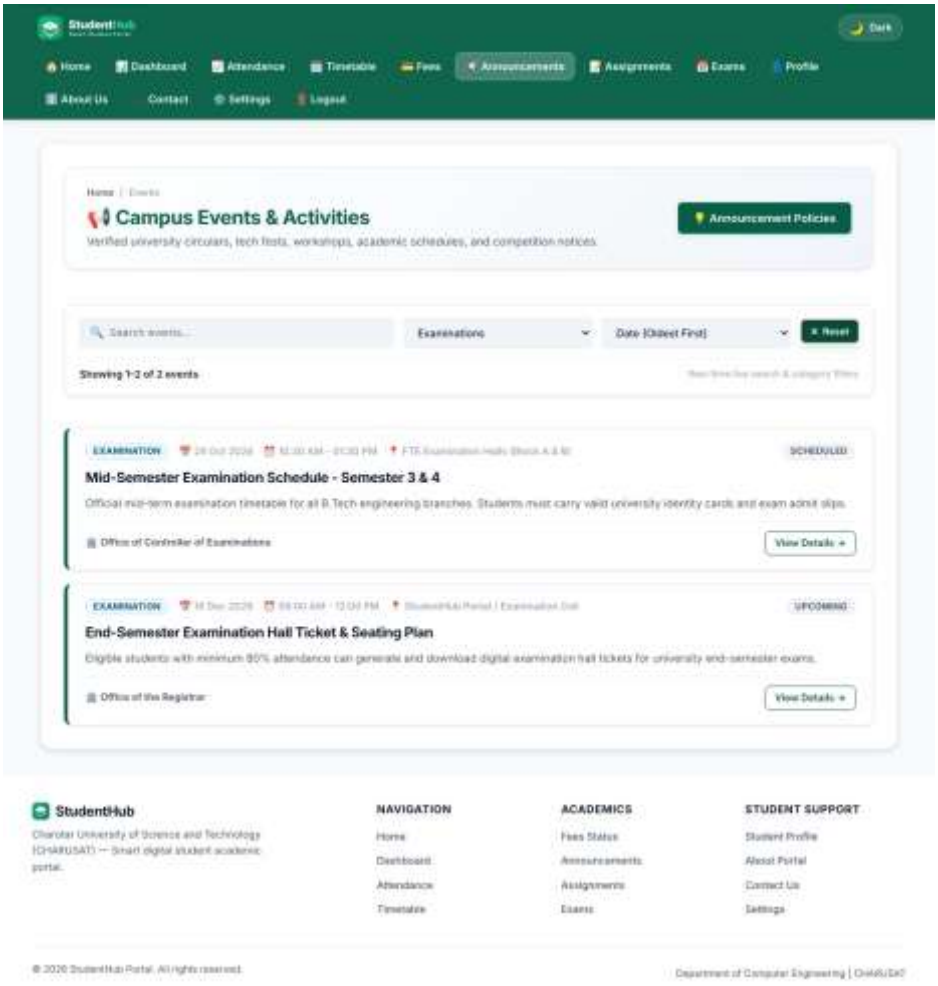
PREVIEW:



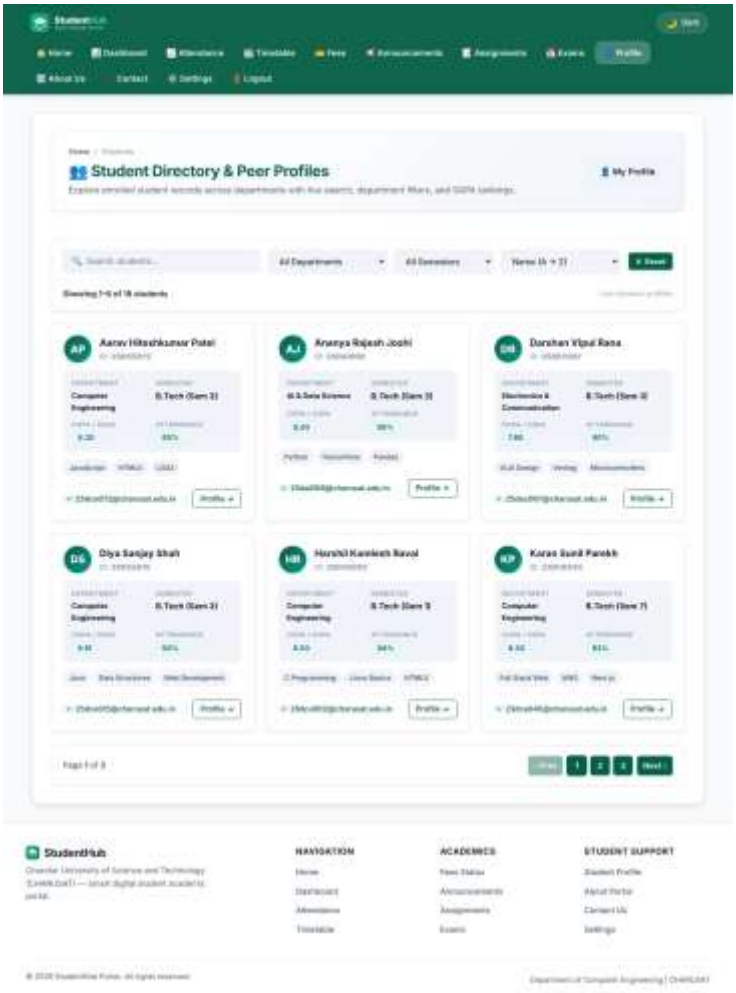
Events page - cards rendered from events.json



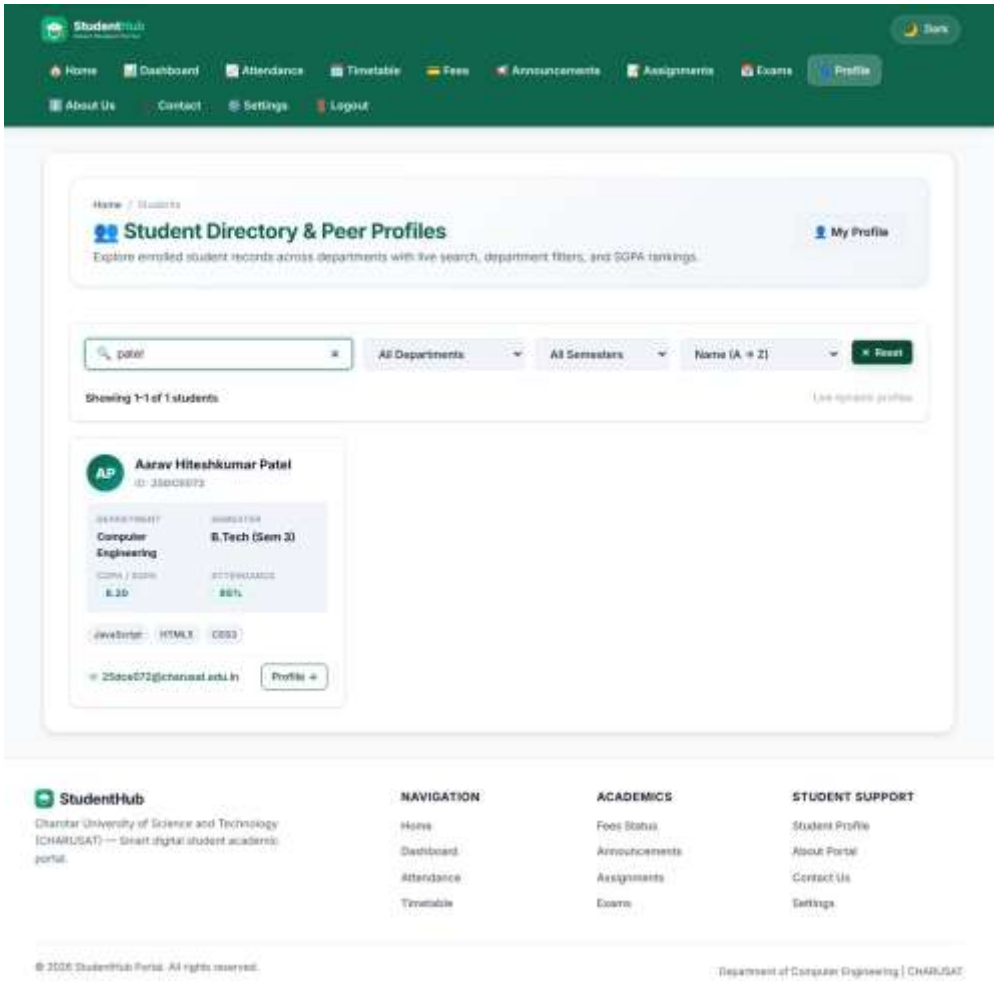
Search "workshop" - results filtered as the user types



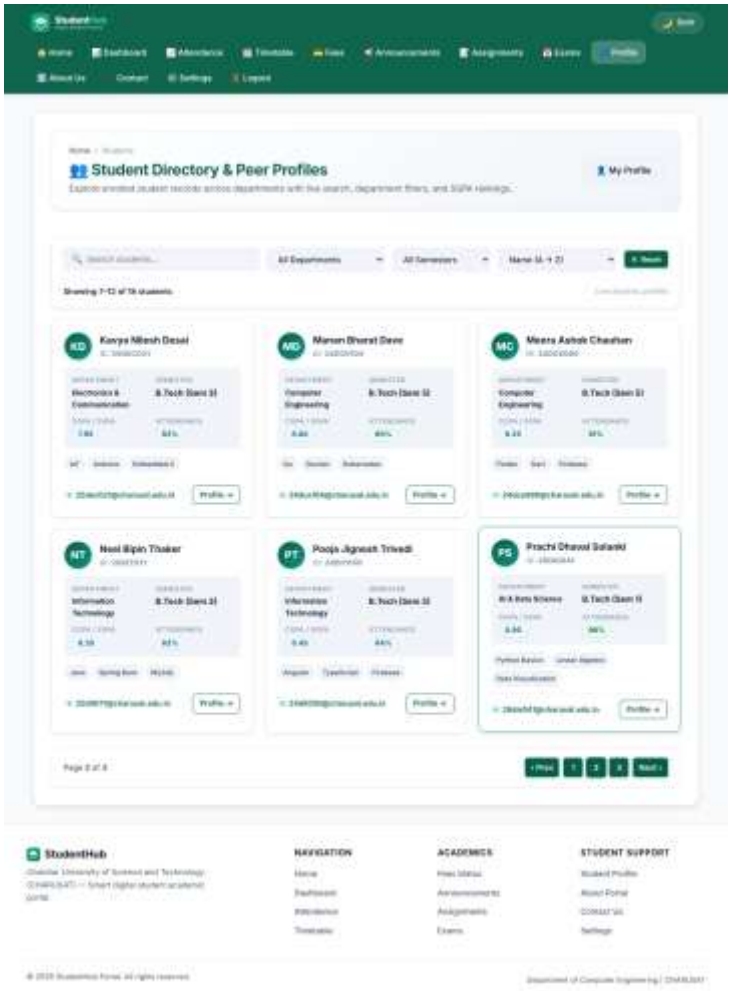
Category filter combined with a sort order



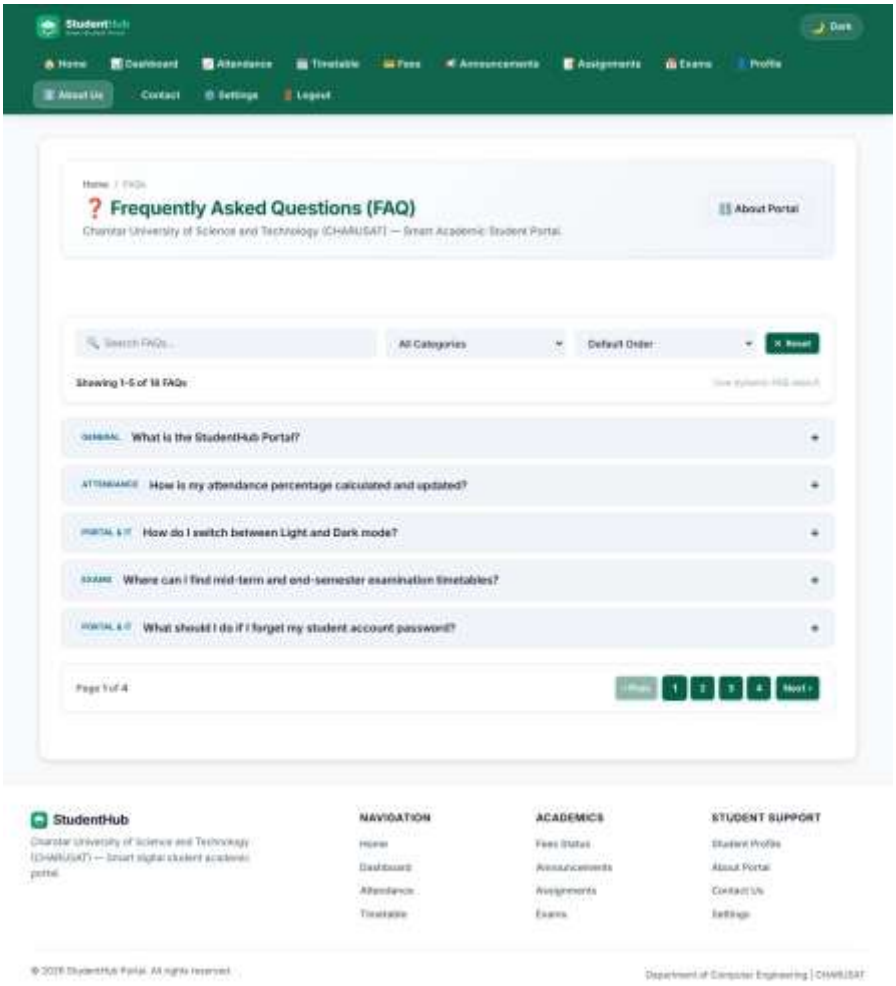
Student directory rendered from students.json



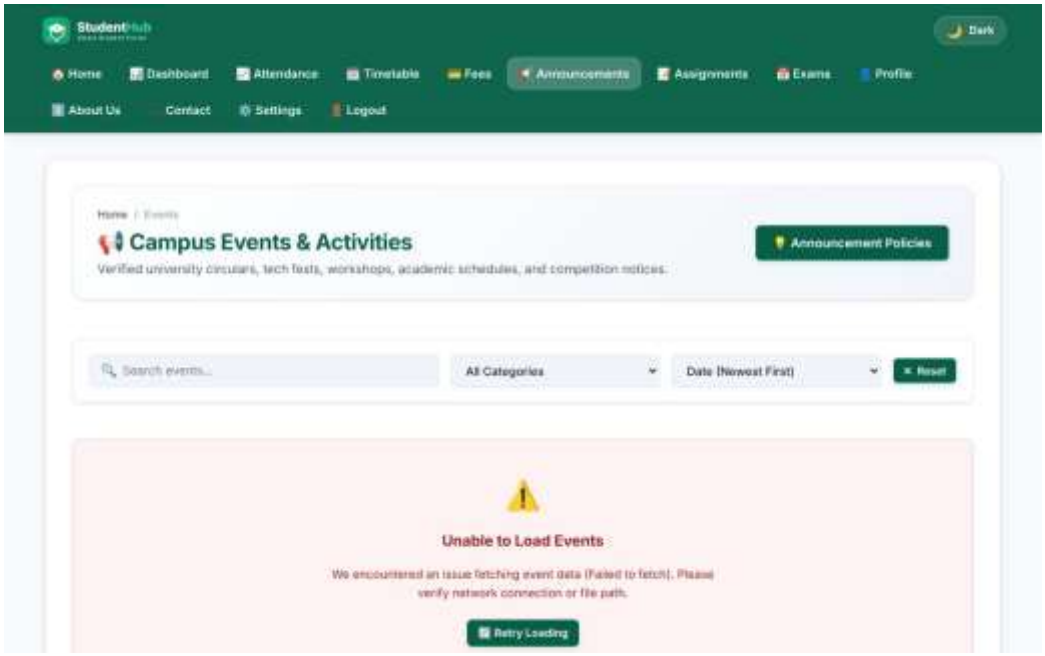
Student search "patel"



Pagination - second page of students



FAQ page rendered from faqs.json



Error state when the JSON request fails (network blocked) with a Retry button

TEST CASES :

#	Input / action	Expected result	Result
1	Open events page	18 events loaded from events.json, loading text shown first	Pass
2	Search "workshop"	Only matching events; count updates (5 events per page)	Pass
3	Category = Workshop + sort	Filter and sort work together	Pass
4	Search text with no match	Empty-state message with a "Clear all filters" button	Pass
5	Students: next page	Next 6 records shown, pagination buttons update	Pass
6	Block the .json request	Error card with a Retry button, no JavaScript error	Pass

KEY QUESTIONS :

1. How is JSON fetched, parsed and rendered?

fetch() is called inside an async function. The response is checked with response.ok, then response.json() parses the body to an array of objects. The array is stored once (allEvents) and every render is built from it with template strings and written with innerHTML after escaping the text.

```
const response = await fetch(url);
if (response.ok) {
  fetchedData = await response.json();
}
...
container.innerHTML = pageItems.map(renderCard).join("");
```

2. Which array methods are used for search, filter, sort and pagination?

- filter() - keeps the items that match the search text and the selected category;
- sort() - orders by date (newest/oldest) or title with localeCompare();
- slice(start, start + perPage) - cuts the current page;
- map() / join() - turns the page items into HTML; forEach() - attaches button listeners.

```
let filtered = allEvents.filter(item => matchesText(item) && matchesCategory(item));
filtered.sort((a, b) => new Date(b.date) - new Date(a.date));
const paginatedItems = filtered.slice(startIndex, startIndex + itemsPerPage);
```

3. How are loading and error states handled?

Before the request a "Fetching ..." block with role="status" and aria-live="polite" is shown. If the request fails or the status is not OK, renderErrorState() shows a message with role="alert" and a Retry button that runs the fetch again. Several relative paths for the JSON file are tried so the page works from different folders. An empty result shows a separate empty-state with a "Clear all filters" button.

4. How is modularity maintained?

Each page has its own file (events.js, students.js, faq.js) wrapped in an IIFE so no variables leak to the global scope; only one object (window.StudentHubEvents = { init }) is exported. Data (JSON), logic (JS), structure (HTML) and style (CSS) are separate files, and the three modules follow the same structure (fetch, process, render, init).